

Clear Path Nexus · RailGuard

Decision support for Indian Railways traffic control: build, verification and readiness report

Generated 07 October 2026, 21:36 UTC · code revision 44bed60 · SEVA 2026 submission

What this is. A controller decision-support system for the whole Indian Railways network built from open data: a digital twin of every station, junction, section and timetabled train; a re-planner that ranks conflict-free alternatives after a disruption; a driver cab advisory; a machine-learning run-time model; a tamper-evident audit trail; and a gateway ready for authorised live feeds.

What it is not. It is advisory. It has no interface to signals, points, interlocking, Kavach/ATP or brakes, and it does not hold or grant movement authority. Authorised signals, the General & Subsidiary Rules and railway staff remain controlling.

How to read the numbers. Everything measured here is a software result on open and synthetic data. None of it is a field measurement on Indian Railways. Readiness percentages are judgements and are stated with their reasons.

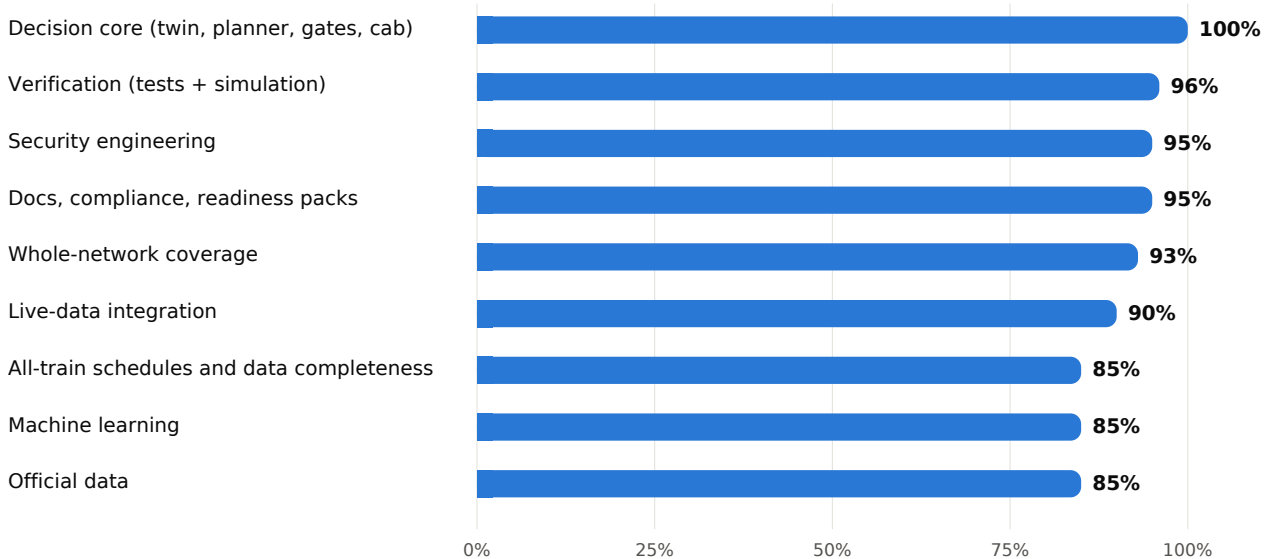
8,990 stations in the twin (all in the open data)	1,454 junctions	8,738 sections between stops	7,580 timetabled train runs in the 2-day window
5,068,492 operations checked in simulation	0 invariant violations on the final code	134 automated tests passing	1.38 vs 1.96 min ML error vs section median (locked test)

1. How much is done, and what is left

Software side: about 92% done - everything this project can build and verify. The rest of the software work needs inputs only Indian Railways holds: CRIS's interface specification, actual running data to retrain on, the official timetable with running days (downloadable only from India), and controller feedback from a trial.

Whole path to railway use: about 52% done, about 48% left. For every railway stage the project has prepared what it can, so that stage becomes review and sign-off. What is left is authorisation, an independent audit, safety acceptance and a field trial. Software alone cannot complete those, so this figure cannot honestly reach 90% before Indian Railways acts.

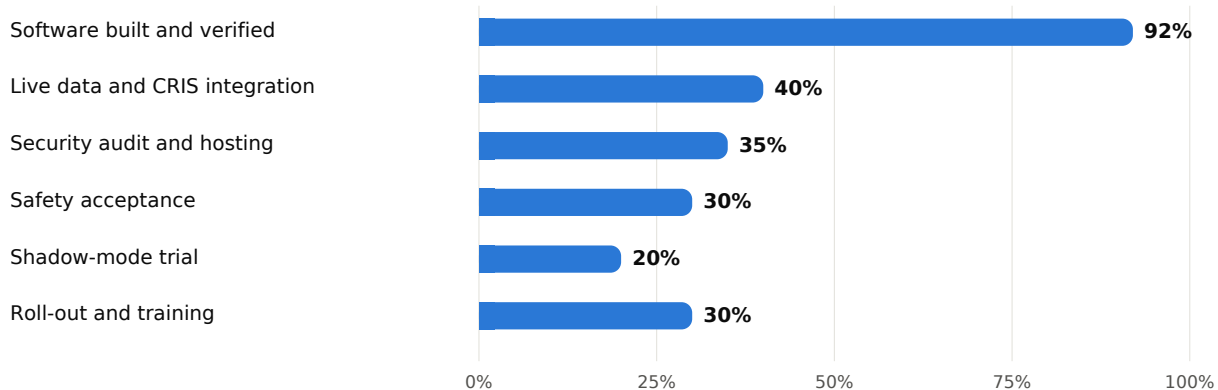
Software readiness by work package (judgement)



Work package	Done	Why this figure
Decision core (twin, planner, gates, cab)	100%	Built; 6.7M checked operations, last defect fixed, final code clean
Verification (tests + simulation)	96%	Unit, API and attack tests; 8 invariants; regression on final code; formal V&V; plan needs IR
Security engineering	95%	Controls, 32 attack tests, SAST/SCA clean, ASVS L2 self-assessment; external audit pending
Docs, compliance, readiness packs	95%	Register, hazard log, safety case, interface contract, trial plan, guide
Whole-network coverage	93%	All 8,990 stations; 914 non-halt placed; 105 stations with suspect coordinates named
Live-data integration	90%	Signed gateway, contract with test vector, IST clock; CRIS field mapping remains
All-train schedules and data completeness	85%	Every train audited; working schedules for 5,187; running days not in open data
Machine learning	85%	Best model on public data; retraining on actual running data needs live feeds
Official data	85%	Pipeline incl. running days, provenance, tests; data.gov.in refuses this build network

Path to railway use (weights are shares of total effort)

Done per stage (judgement)



Stage	Weight	Done	Done by this project	Left: only the railway can do
Software built and verified	35%	92%	This repository	-
Live data and CRIS integration	15%	40%	Gateway, contract, test vector, simulator	Authorise access; share spec; issue keys; test in CRIS environment
Security audit and hosting	10%	35%	ASVS self-assessment, attack tests, SAST/SCA	CERT-In/STQC audit; host on IR infrastructure
Safety acceptance	15%	30%	Draft hazard log and safety case with evidence	Hazard workshop; scoring; independent assessment; RDSO/IR acceptance
Shadow-mode trial	20%	20%	Trial tooling (/shadow), protocol, metrics	Run 3-12 months on a division; review disagreements
Roll-out and training	5%	30%	Operator guide and training modules	Deliver training; issue procedures

Readiness packs: seva2026/railway_readiness/ - hazard log, safety case, live-data interface contract, ASVS checklist, shadow-trial plan, operator guide.

2. What was built

Component	What it does
National twin (railguard/national.py)	Every station, junction and section from the open timetable; 7,580 train runs over a two-day window; occupancy index for fast conflict search; disruption, ranking, approval, threats, cab, replay.
Tabletop twin + TwinTrack (engine.py, hardware/)	A 10-section demonstration line with two trains, ESP32 sensor/cab firmware and six scripted scenarios.
Planner and scoring (planner.py, scoring.py)	Generates CONTINUE / HOLD / PATH-THROUGH / PRIORITY (with cascading re-paths) / REROUTE options, resolves conflicts by making lower-priority trains yield, ranks by 7 explainable factors.
Evidence gate and approval	Fresh/aging/stale evidence states; a named controller approves only the latest recommendation for the current state; anything older is superseded.
Threat registry	Converging paths, opposing occupancy, blocked/closed sections, stale data, route deviation, impossible movement; OPEN → ACKNOWLEDGED → CLEARED lifecycle.
Nexus Control / Nexus Cab (static/)	Controller console (tabletop and national map) and driver cab view; strict CSP, no inline code, tokens never in URLs sent to the server.
Audit (audit.py)	Hash-chained events with optional HMAC, persisted as JSON lines, verified by CLI; decision snapshots that replay to the same ranking.
Run-time ML (runtime_model.py, training.py)	Gradient-boosted trees with quantile intervals and conformal calibration; measured training rounds with a locked test set.
Official data (official.py)	Registry, licences, SHA-256 provenance and a parser for the data.gov.in timetable; official rail distances replace straight-line estimates when present.
Live-feed gateway (livefeed.py)	Signed batches from authorised sources (RTIS/NTES/COA-style); replay protection; GNSS map-matching; station events become disruptions for the controller.
Security layer (security.py)	Roles, fail-closed production mode, rate limits, body caps, host allow-list, strict request schemas, security headers.
Working schedules (schedules.py)	Every train's stop matrix (day, times, dwell, distance, section speed), per-train metrics and a completeness audit of all 5,208 trains; running days honoured when official data supplies them.
Shadow trial (shadow.py)	Logs controllers' actual decisions beside the recommendations shown at the time and measures agreement, without changing anything: the evidence a field trial needs.
Simulation harness (simulate.py)	Randomised operation sequences on both twins with 8 safety invariants checked after every operation.
Assistant (assistant.py, free_assistant.py)	Offline or local open-source model (Ollama) by default; the paid Claude option is opt-in only.

Technology used

Area	Technology
Language / runtime	Python 3.11 (CI) and 3.13; HTML, CSS and plain JavaScript (no framework); Arduino C++ for the ESP32
Service	FastAPI 0.142, Starlette 1.7, Pydantic 2 (strict models), Uvicorn
Data	SQLite, pandas, NumPy; DataMeet open timetable (CC0); data.gov.in pipeline (GODL)
Machine learning	scikit-learn HistGradientBoosting (absolute-error and quantile losses), GroupKFold, split-conformal calibration, joblib
Security	HMAC-SHA256, SHA-256 hash chains, constant-time comparison, token buckets, CSP; Bandit (SAST), pip-audit (SCA)
Quality	pytest, ruff 0.8.6 (pinned, as in CI), radon, vulture, GitHub Actions CI, Playwright + Chromium for UI checks
Assistant	Ollama (local, free) or the Anthropic SDK (opt-in, paid)
Report	ReportLab (this PDF)

3. Data: the whole network

The twin is built from the open Indian Railways timetable (DataMeet, CC0): 8,990 stations, 5,208 trains and 417,080 stop rows, pinned by SHA-256. Every attribute that the open data does not publish is inferred and labelled with how it was obtained, so nobody mistakes an inference for a survey.

Quantity	Value	How it was obtained
stations total	8,990	every station in the open data
stations	7,679	stations where at least one train halts
stations without halt placed on sections	914	non-halt stations projected onto the section they lie on (≤ 2 km)
stations without halt not placed	397	no coordinates, or farther than 2 km from every section
junctions	1,454	stations joined to 3 or more neighbours
sections	8,738	pairs of consecutive stops
sections inferred multi track	5,098	opposing trains timetabled on it at the same time
sections assumed single	3,640	no such evidence: treated as single line (the safe assumption)
runs in window	7,580	train numbers x start days in 2 days
occupations	843,568	section occupations in the window

Official data. `india_rail/official.py` registers the Government sources (`data.gov.in` timetable, Trains at a Glance, station codes) with publisher and licence. It parses the `data.gov.in` format, including its 00:00:00 placeholders, and records each file's SHA-256 in `official_provenance`. It reconciles the file against the community data and gives the twin official rail distances per section. The Government portals reset connections from this cloud build network, so the file must be fetched from India with `python -m india_rail official --file <csv>`. Tests cover the whole path with a format fixture.

Known limits. The timetable snapshot is from about 2016 and has no running days, so trains without known days are treated as daily (the twin honours running days as soon as an official file supplies them). Track counts, line speeds and lengths are inferred. Loops, platforms and signals are not in any open dataset.

All trains: working schedules and completeness

Every published train was checked: `python -m india_rail schedules --audit`. Each train's working schedule (every stop with day, arrival, departure, dwell, distance from origin, section run time and speed) is available from `python -m india_rail schedules --train <number>` and `GET /trains/<number>/working-schedule`.

Measure	Value
Trains in the published list	5,208
With a usable timetable	5,187 (99.6%)
Timed stops / halts with dwell	394,125 / 93,620
Timetable duration within 30 min of the published duration	5,129 trains (median difference 0.0 min)
Published duration / distance known	99.71% / 99.19%
Accommodation classes / return train known	45.87% / 88.5%
Running days known	0.0% (not in the open data)
Stations with suspect published coordinates	105 (e.g. KRLS 91, GOH 91, KTHL 87, MIR 87 impossible sections)

Flag	Trains
RUNNING_DAYS_UNKNOWN	5,208
NO_CLASS_INFORMATION	2,819
STOPS_WITHOUT_COORDINATES	1,071
DISTANCE_INCONSISTENT_WITH_TIME	794

Flag	Trains
DURATION_DIFFERS_FROM_PUBLISHED	50
NO_PUBLISHED_DISTANCE	42
RETURN_TRAIN_NOT_LISTED	38
NO_USABLE_TIMETABLE	21
NO_PUBLISHED_DURATION	8

4. Machine learning: section run-time model

The model predicts the scheduled run time between two stops: the planning quantity a re-planner needs. It learns the correction to a section prior. Section priors are recomputed inside every cross-validation fold from training trains only, and folds are grouped by train pair, so no evaluated train is ever seen in training.

Training protocol. Before any training round, 15% of train groups were locked away as a final test set (58,110 sections, 767 trains). Each round changed one thing and was kept only if 3-fold grouped CV error improved by at least 0.005 min. Training stopped when a full pass found no further gain.

Cross-validated mean absolute error by round (minutes; lower is better)



Round	Change	Kept	CV MAE (min)	Within 2 min
0	baseline (current production configuration)	yes	1.869	76.0%
1	round predictions to whole minutes (timetables are minute-resolution)	yes	1.857	81.7%
2	type- and direction-specific section priors, run-time spread on the section	yes	1.707	83.6%
3	local pace from the two sections either side (existing trains)	yes	1.460	87.4%
4	start from the type-specific section prior	yes	1.452	87.3%
5	more trees, slower learning	no	1.448	87.4%
6	larger trees	yes	1.420	87.7%
7	smaller leaves, more regularisation	no	1.425	87.7%
8	blend toward well-supported section medians (k=20)	no	1.734	84.3%
9	blend toward well-supported section medians (k=60)	no	1.568	86.4%
10	more trees, slower learning	yes	1.415	87.8%
11	smaller leaves, more regularisation	no	1.419	87.7%
12	blend toward well-supported section medians (k=20)	no	1.732	84.3%
13	blend toward well-supported section medians (k=60)	no	1.566	86.5%
14	smaller leaves, more regularisation	no	1.419	87.7%
15	blend toward well-supported section medians (k=20)	no	1.732	84.3%
16	blend toward well-supported section medians (k=60)	no	1.566	86.5%

Locked test set, scored once

Predictor	MAE (min)	Within 2 min	MAPE ≥ 10 -min sections
Final model	1.378	88.1%	16.0%
Section-median baseline	1.957	80.1%	24.4%
Speed-by-train-type baseline	2.611	63.3%	30.7%

Production model (5-fold grouped CV on all data)

Mode	MAE (min)	Within 2 min
Existing train (uses its other sections)	1.387	88.0%
New path (no train context)	1.680	83.7%
Section-median baseline	1.953	80.4%

P10-P90 interval coverage after conformal calibration: 79.99% (target 80%), mean width 3.855 min.

Why not 'perfect'. The target is a published timetable in whole minutes with operator-chosen allowances that no public feature explains, so zero error is not achievable. The next real gain needs actual running data (NTES/COA), which is exactly what the live-feed gateway is for.

5. Simulation: 5 million checked operations

An episode starts a fresh twin with random conditions, then runs a random sequence of operations. The operations are clock ticks, recommendations, approvals (including stale and wrong ones), section and sensor events, malformed or impossible feed data, holds and acknowledgements. Eight safety invariants are checked after every operation, and every episode is seeded so any failure can be replayed exactly. Full end-to-end episodes cost about 0.1 s each, so the 5-million target is counted honestly as checked operations, not as episodes.

Result. The main run checked over 5 million operations. The national twin had no violations. The tabletop twin had 8 violations in 70,000 episodes: 7 were one real defect (re-holding a train that was already held released it forward) and 1 was a test-case error in the harness. Both were fixed, all 8 episodes replay clean, and a further run on the fixed, final code is reported below.

Twin	Episodes	Operations checked	Violations	CPU ms / episode
demo	70,000	4,018,172	8	135.303
national	90,000	1,050,320	0	104.996
Total	160,000	5,068,492	8	

Simulated 5-second steps (tabletop twin): 55,375,197. Wall time 105 min on 3 workers. Code checksum 52f194c00c9a6c06...

Regression run (after the main run's code): 12,000 episodes, 411,434 operations, 0 violations (code checksum 34e99e011fc49a26...).

Run on the exact final code: 24,000 episodes, 1,201,001 operations, 0 violations (code checksum 752bfcc1c5691bb8...).

All runs together: 6,680,927 checked operations, 8 violations.

Invariants

Invariant	Meaning
SAFE_SEPARATION	Approved plans never conflict; simulated trains never share a single-line section or close up on one track; later conflicts entering the planning horizon are always on the threat list
APPROVAL_GATE	Only the latest recommendation for the current state can be approved, with fresh evidence, no open critical threat, and no closed or obstructed section on the plan
EVIDENCE_GATE	Stale or missing mandatory evidence never yields an approvable recommendation
INPUT_REJECTION	Malformed, replayed, future or physically impossible observations never move the twin
CAB_ADVISORY	No speed band without fresh evidence and an approved plan; never a proceed status with a blocked section ahead; band never above line speed
RANKING	Ranked plans are conflict-free, feasible, monotonic in time, never early, scored 0-1, winner first; yields never re-route a train
REPLAY	Snapshots verify and replay to the same ranking; the audit chain verifies
NO_CRASH / LIVENESS	No unexpected exception; every episode completes within 60 s

What the simulation found (all fixed, each with a regression test or replayable episode)

#	Finding	Fix
1	An approval went through after the state changed at the same clock time (e.g. a controller hold), and cancelled the hold	Every state change bumps a version; approval must match the ranked version
2	A train re-planned mid-section was moved back to the start of the section	Plans carry the starting offset
3	A held train could stop at the start of the next section instead of the station	Holds stop at the node
4	A closed section ahead raised no threat on the tabletop twin (cab showed NORMAL)	SECTION_CLOSED critical threat
5	Rounded plan times put a re-planned train back at the station for a fraction of a minute	Position held until the plan's first time
6	National planner checked closures and conflicts only from the disruption station onward	Planning starts at the first section the train can still change
7	PRIORITY-path option skipped the closed/obstructed/axle-load check	One feasibility filter for every option
8	Conflicts on the section a train already occupies were never examined	Conflict search starts at the occupied section
9	Two re-pathed trains could ping-pong forever (planning hang)	Bounded cascade; LIVENESS invariant
10	The 240-min horizon was applied per train, so a pair was seen from one side only	Pair-symmetric horizon
11	The PRIORITY option returned a plan that its own re-check had not validated	Return the validated plan
12	Replay did not reproduce rankings (threats/acks not restored; order-dependent index)	Restore derived state; sorted index; 1,585 replays 0 mismatches
13	A disruption reported by a live station event could not be applied where the train really was	Feed events pin the stop index
14	Main run, 7 episodes: re-holding an already-held train released it towards its timetabled position	A re-hold never moves the stop point later; regression test fails on the old code
15	Main run, 1 episode: a harness test case (not the system) mislabelled a plausible move as a jump	Harness fixed

6. Security

Check	Result
Adversarial API tests	32 cases pass: auth bypass, token tricks, path traversal, NaN/Infinity, type confusion, JSON bombs, injection strings, method tampering, CORS, error leakage, role separation
Security tests	Fail-closed production mode, role separation, host allow-list, body limits, security headers, rate limits, bounded memory, audit tamper detection
Live-feed tests	Bad signature, unknown key, replayed nonce, stale timestamp, sequence regression, oversize batch, off-route fixes
Static analysis (Bandit)	0 findings (7 reviewed suppressions, each justified in the code)
Dependency audit (pip-audit)	0 known vulnerabilities in 45 pinned packages
Browser	Strict Content-Security-Policy verified in Chromium: no console errors on any page

Security findings fixed during the audits

#	Finding	Fix
1	Starlette 0.41.3 had 7 published advisories; pytest 8.3.4 had 1	FastAPI 0.142.2 / Starlette 1.7.0 / pytest 9.0.3; pip-audit 0
2	NaN in a request crashed the 422 response (500) and errors echoed client input	Validation handler that never echoes input
3	Lax coercion accepted "5" and true as numbers and ignored unknown fields	Strict request models: exact types, no extra fields, finite numbers
4	Audit chain could not be verified after a restart (second GENESIS in one file)	Chain resumes from the file
5	Ollama URL from the environment was opened without a scheme check (file://)	http(s) only
6	SQL assembled with an f-string (safe but fragile)	Constant statement, all values bound
7	Chunked uploads bypassed the body-size cap	411 for bodies without a length
8	Interactive API docs exposed in production	Disabled when RAILGUARD_MODE=production

7. Five audit passes

Each pass looked at the whole system through one lens, fixed what it found, and re-ran the full test suite and lint before the next pass. Details are in [seva2026/AUDIT_LOG.md](#).

Pass	Lens	Main findings and fixes	Evidence
1	Safety logic	14 system defects found by the randomised simulation and fixed (section 5): stale approvals, position snapping, holds and re-holds, closures, planner scope, cascade hang, horizon symmetry, replay determinism	simulate.py; 0 violations on the final code
2	Security	8 findings fixed (section 6): dependency CVEs, NaN crash and input echo, type coercion, audit restart, URL schemes, SQL construction, chunked bodies, docs exposure	test_attacks.py, Bandit 0, pip-audit 0
3	Performance	Plan and stress caches keyed on exact inputs, cheap snapshot copies, distance cache: 39% less CPU per episode with byte-identical outputs (6 scenario and 30 episode checksums equal)	baseline checksums, profiles
4	Data and ML	Locked-test training protocol; model error 1.96 → 1.38 min (-30%); production model retrained; official-data pipeline; 914 non-halt stations placed; placeholder-time bug caught by tests	training_log.json, test_official.py
5	Code quality	Dead code removed; national builder split (complexity F 62 → C 16) with an identical data checksum; lint/format clean on Python 3.11 and 3.13; CI parity checked	radon, vulture, ruff, CI

8. Ready for live data

When the Ministry of Railways / CRIS authorises a feed, it plugs in without code changes to the twin.

- Envelope: source, key id, sent-at, nonce, sequence and events, HMAC-SHA256 signed with a per-source key (RAILGUARD_FEED_KEYS). Refused before reading: bad signature, unknown key, clock skew beyond ± 120 s, replayed nonce, non-increasing sequence, more than 250 events.
- POSITION events (e.g. RTIS GNSS) are map-matched onto the train's planned route within 3 km. Anything else is raised as a route-deviation threat, never silently accepted.
- STATION events (e.g. NTES/COA arrival/departure) update the position. Lateness of 5 minutes or more is recorded as a disruption for the controller to decide on. The feed approves nothing.
- With every involved train on a fresh live fix, recommendations become REVIEWABLE instead of PLANNING_ONLY. This is tested end to end with a signed feed simulator.
- Remaining for go-live: CRIS's interface specification (field names, identifiers, transport), credentials, and a shadow-mode trial.

9. Legal, safety and data compliance

Instrument	How the design addresses it	Status
Railways Act 1989; General & Subsidiary Rules	Advisory only; named-controller approval; no interface to the safety chain	Met by design
RDSO / EN 50126, 50129, 50716; CRS	Outside the SIL chain by design; V&V; evidence prepared; safety case and assessor are IR steps	Needs authority
IT Act 2000 s.43/66/70 (CII)	Ingests only signed, pushed, authorised data; no scraping or probing	Met by design
CERT-In Directions 2022	Tamper-evident persistent logs (operator keeps ≥ 180 days in India), NTP-synced clocks, 6-hour incident runbook	Ready (operator config)
DPDP Act 2023 / Rules 2025	No passenger data; only controller IDs for accountability	Met by design
GODL / NDSAP, CC0, ODbL	Publisher, licence and SHA-256 recorded for every source	Met
CERT-In/STQC audit, GIGW	Self-tested; needs an empanelled auditor	Open (external)

Full register: [seva2026/COMPLIANCE_REGISTER.md](#). This is an engineering register, not legal advice.

10. What is left

- Indian Railways / CRIS: authorise live feeds (NTES/RTIS/COA) and share the interface specification.
- Download the official data.gov.in timetable from a connection in India and ingest it (one command).
- Retrain the run-time model on observed running data, and add a delay-propagation model once real delays are available.
- Independent CERT-In/STQC security audit; hosting on railway infrastructure in India.
- Safety case and hazard log signed by IR; independent safety assessment; RDSO acceptance of the advisory role.
- Shadow-mode trial on one division: compare recommendations with controllers' actual decisions.

11. How to run

Task	Command (in india_rail_ai/)
Install	<code>pip install -r requirements.txt</code>
Get data and train	<code>python -m india_rail setup</code>
Serve the API and consoles	<code>python -m india_rail serve → /control, /control/national, /cab</code>
Tests and lint	<code>python -m pytest -q; ruff check .</code>
Simulation	<code>python -m india_rail.railguard.simulate --demo 70000 --national 90000</code>
ML training rounds	<code>python -m india_rail.training</code>
Official data	<code>python -m india_rail official --file timetable.csv</code>
All-train schedules	<code>python -m india_rail schedules --audit --train 12951 --export DIR</code>
Shadow trial	<code>POST /railguard/national/shadow/actual; GET /railguard/national/shadow/report</code>
Verify an audit log	<code>python -m india_rail.railguard.audit verify</code>
Rebuild this report	<code>python scripts/build_report.py</code>